

Implémentation d'une file bornée

0.1 Définition et principe

Une *file* est une structure de données FIFO (*First In, First Out*) : le premier à entrer est le premier à sortir.

Il est *préférable* que toutes les opérations usuelles sont en temps constant :

Opération	Complexité
$\Theta(1)$ defiler	$\Theta(1)$ estVide
$\Theta(1)$ estPleine	$\Theta(1)$ height

Il s'agira dans cette fiche de révision de présenter les différentes manières de les implémenter requises en MP2I.

1 Implémentation par file bornée et tableau statique

Dans le cas d'une **file bornée**, la capacité maximale est fixée à l'avance. On peut alors utiliser un **tableau statique** et l'exploiter de manière **circulaire** afin d'éviter les décalages coûteux.

Structure de la file

```
const int TAILLE = 10; // On prends 10 comme exemple, peut etre
                          un autre entier

typedef struct {
    int* tab;
    int tete;
    int queue;
    int taille;
} fileStat;
```

Initialisation

```
void creerVide(file* f) {
    f->tete = 0;
    f->queue = 0;
    f->taille = 0;
}
```

Tests d'état

```
bool estVideStat(fileStat *f) {
    return f->taille == 0;
}

bool estPleineStat(fileStat *f) {
    return f->taille == TAILLE;
}
```

Ajout d'un élément

```
void enfilerStat(fileStat *f, int e) {
    assert(!estPleineStat(f));

    f->tab[f->queue] = e;
    f->queue = (f->queue + 1) % TAILLE;
    f->taille++;
}
```

Retrait d'un élément

```
int defilerStat(fileStat *f) {
    assert(!estVideStat(f));

    int val = f->tab[f->tete];
    f->tete = (f->tete + 1) % TAILLE;
    f->taille--;

    return val;
}
```

Le compteur `taille` est essentiel : sans lui, la condition `[ tete == queue ]` serait ambiguë, correspondant à la fois au cas d'une file vide et au cas d'une file pleine.

L'introduction de `taille` permet de lever cette ambiguïté de manière simple.

L'implémentation d'une file bornée par tableau circulaire permet :

- d'éviter toute allocation dynamique,
- de garantir des opérations en temps constant,
- de disposer d'invariants simples et stables.

2 Implémentation naïve par maillons chaînée

Si cette fois-ci, on ne connaît pas à l'avance le nombre maximal d'éléments que peut contenir notre liste, alors on peut utiliser une liste chaînée afin d'avoir une structure modulable

Structures Maillon & file

```
typedef struct sMaillon1 {
    int data;
    struct sMaillon1 *suivant;
} maillon;

typedef struct sFile {
    maillon *tete;
    maillon *queue;
} file;
```

Il est nécessaire d'utiliser l'interface `file` ici pour maintenir une complexité en $\Theta(1)$ dans les algorithmes de base en utilisant les pointeurs définis vers les deux extrémités de la file.

Initialisation

```
file *creerVide() {
    file *f = malloc(sizeof(file));
    f->tete = NULL;
    f->queue = NULL;

    return f;
}
```

Ce sont les conditions normales pour une liste vide.

Tests d'état

```
bool estVide(file *f) {
    return f->tete == NULL;
}
```

Ajout d'un élément

```
void enfiler(file *f, int e) {
    maillon *nouveau = malloc(sizeof(maillon));
    nouveau->data = e;
    nouveau->suivant = NULL;

    if (estVide(f)) {
        f->tete = nouveau;
        f->queue = nouveau;
        return;
    }

    f->tete->suivant = nouveau;
    f->tete = nouveau;
}
```

⚠ Bien traiter les cas où la liste est vide

Retrait d'un élément

```
int defiler(file *f) {
    assert(!estVide(f));

    maillon *tmp = f->queue;
    int res = tmp->data;

    if (tmp->suivant == NULL) {
        f->queue = NULL;
        f->tete = NULL;
    } else {
        f->queue = f->queue->suivant;
    }

    free(tmp);
    return res;
}
```

⚠ Bien traiter le cas où la file sera nulle juste après i.e. la file ne contient qu'un unique élément.

3 Implémentation avec une `list` en Caml